

Assessment: Backend Dependencies, Git, and GitHub

1. Exploring Backend Dependencies Used in Applications

Introduction

Backend development is the part of an application that works behind the scenes. It handles business logic, databases, authentication, APIs, and communication between different services. To build backend applications efficiently, developers use various dependencies and libraries instead of writing everything from scratch.

Backend dependencies are external packages, frameworks, or tools that provide pre-built functionality to simplify development.

What Are Backend Dependencies?

Backend dependencies are software packages that an application relies on to perform specific tasks. These dependencies are installed and managed using package managers.

Examples:

- Python: pip
- Node.js: npm
- Java: Maven and Gradle
- .NET: NuGet

Instead of writing thousands of lines of code, developers can use dependencies to add functionality quickly and efficiently.

Common Backend Dependencies

1. Web Frameworks

Frameworks help developers build APIs and web applications faster.

Examples:

- FastAPI (Python)
- Flask (Python)
- Django (Python)
- Express.js (Node.js)

- Spring Boot (Java)

Purpose

- Handle HTTP requests and responses.
 - Create REST APIs.
 - Manage routing and middleware.
-

2. Database Dependencies

Applications need databases to store and retrieve information.

Examples:

- SQLAlchemy
- Prisma
- Sequelize
- Mongoose

Purpose

- Database connection management.
 - Query execution.
 - Data modeling.
-

3. Authentication Dependencies

Authentication libraries secure applications and verify users.

Examples:

- JWT (JSON Web Tokens)
- OAuth Libraries
- Passport.js

Purpose

- User authentication.
 - Authorization.
 - Access control.
-

4. AI and Machine Learning Dependencies

Modern applications increasingly integrate AI capabilities.

Examples:

- OpenAI SDK
- LangChain
- Hugging Face Transformers
- TensorFlow
- PyTorch

Purpose

- AI model integration.
 - Natural Language Processing.
 - Generative AI applications.
-

5. API Communication Dependencies

Applications often communicate with external services.

Examples:

- Requests (Python)
- Axios (JavaScript)
- HTTPX

Purpose

- Calling APIs.
 - Sending requests.
 - Receiving responses.
-

Dependency Management

Developers maintain dependencies through configuration files.

Examples:

Python

requirements.txt

Node.js

package.json

These files contain all required libraries and their versions.

Benefits

- Easy project setup.
- Consistent development environment.

- Simplified deployment.
-

Importance of Backend Dependencies

Backend dependencies help developers:

- Reduce development time.
 - Improve application security.
 - Increase reliability.
 - Access advanced features quickly.
 - Maintain scalable applications.
-

Conclusion

Backend dependencies are essential components of modern software development. They provide ready-made functionality for web development, database management, authentication, AI integration, and API communication. Proper dependency management helps build secure, scalable, and efficient applications.

2. Git and GitHub: Usage, Push and Pull Methods

Introduction

Git is a distributed version control system used to track changes in source code. GitHub is a cloud-based platform that hosts Git repositories and enables collaboration among developers.

Together, Git and GitHub help teams manage code efficiently and work on projects collaboratively.

What is Git?

Git is a version control system created to track code changes over time.

Features

- Tracks code history.
 - Supports collaboration.
 - Allows rollback to previous versions.
 - Enables branching and merging.
-

What is GitHub?

GitHub is an online platform that stores Git repositories.

Features

- Cloud-based repository hosting.
- Team collaboration.
- Pull requests.
- Issue tracking.
- CI/CD integration.

Git manages code locally, while GitHub stores code remotely.

Git Workflow

A typical Git workflow involves:

1. Create or modify files.
 2. Stage changes.
 3. Commit changes.
 4. Push changes to GitHub.
 5. Pull updates from GitHub.
-

Common Git Commands

Initialize Repository

```
git init
```

Creates a new Git repository.

Check Status

```
git status
```

Displays modified and untracked files.

Add Files

```
git add .
```

Stages files for commit.

Commit Changes

```
git commit -m "Added new feature"
```

Saves changes locally.

Git Push

Push uploads local commits to GitHub.

Command

```
git push origin main
```

Purpose

- Upload code to GitHub.
- Share changes with team members.
- Backup project code.

Example

After completing a feature, developers push the latest code to the remote repository.

Git Pull

Pull downloads the latest changes from GitHub.

Command

```
git pull origin main
```

Purpose

- Synchronize local repository.
- Get teammates' updates.
- Prevent conflicts.

Example

Before starting work, developers usually pull the latest code from GitHub.

GitHub Copilot and Git

GitHub Copilot is an AI-powered coding assistant developed by GitHub and OpenAI.

Uses of GitHub Copilot

- Code generation.
- Code completion.
- Documentation assistance.
- Bug fixing suggestions.

- Unit test generation.

Workflow with Copilot

1. Developer writes code.
 2. Copilot suggests completions.
 3. Developer reviews suggestions.
 4. Code is committed using Git.
 5. Changes are pushed to GitHub.
 6. Team members pull updates.
-

Push and Pull Workflow Example

Step 1

Create a new file.

Step 2

Stage changes.

```
git add .
```

Step 3

Commit changes.

```
git commit -m "Added login functionality"
```

Step 4

Push changes.

```
git push origin main
```

Step 5

Another developer updates the repository.

Step 6

Pull latest changes.

```
git pull origin main
```

This ensures everyone works with the latest version of the project.

Benefits of Git and GitHub

- Version control.
 - Team collaboration.
 - Code backup.
 - Change tracking.
 - Branch management.
 - AI-assisted development with GitHub Copilot.
-